Month 2 progress report

Baakhapaa: AI-powered pre-production intelligence system

Weeks 5 to 8 of 12 · Prepared for Baakhapaa · Reference: proposal §8

All four Month 2 deliverables are complete and were verified again this month. Because they had been built ahead of schedule during Month 1, the four weeks went instead on the work that decides whether any of it survives a real user: testing what had never been tested, running the system against live providers for the first time, and measuring the feature the product is built around.

That measurement is the most important result of the month, and it is not a good one. Retrieval ranks the correct craft technique first in **20%** of the queries the editor actually sends. The figure is reported here rather than buried because the first attempt to measure it produced a flattering 82.4%, and understanding why that was wrong is worth more than the number itself.

1. What the month produced

Assigned deliverables	Four, all complete, all re-verified
Tests added	104 backend, 26 frontend components brought under test
Suite size	765 backend tests in 42 files; 1,020 frontend tests in 54 files
Defects found and fixed	2 security, 5 from writing a screenplay, 1 in the test suite itself
First measured baseline	20% precision@1 on retrieval
Cost model	Corrected by a factor of twelve
Still not done	Deployment, a cloud database, any real payment

2. Measuring retrieval for the first time

Retrieval is the mechanism the whole product rests on. It searches a library of screenwriting techniques and inserts the closest matches into the AI prompt, and it is why the system's advice is specific rather than the generic guidance any language model would give. It had never been measured.

A test set of 34 cases was built from the craft library itself and scored on precision@1 (whether the correct technique ranked first), precision@3, and mean reciprocal rank. The first run reported 82.4% precision@1.

That number was wrong, and the reason is instructive. The test set contained two kinds of case with very different difficulty:

```
REAL QUERIES (what the editor actually sends, n=5)
  precision@1  20.0%     precision@3  60.0%

SANITY CHECK (an entry finds itself, n=29)
  precision@1  93.1%     <- should stay near 100%
```

Twenty-nine of the cases search using text that was itself used to build the index, so finding the right answer is close to guaranteed. Only five resemble what the application really sends when a writer presses a button. Averaged together, twenty-nine easy cases drowned five real ones, and the headline read four times better than the truth.

The harness now reports the two groups separately and leads with the real queries. The self-retrieval group is kept, but as a sanity check rather than a score: if it ever falls far below 100%, something is broken in embedding or storage, which is a different question from whether retrieval is any good.

The baseline is 20%, and it is now committed. It agrees with four checks that already existed inside the corpus loader, which had been printing "OK" on every run while ranking the wrong craft level first in three cases out of four. Nobody had looked at what they were actually reporting.

The breakdown gives the first lead for improving it: dialogue is the weakest craft level at 57%, and entries tagged as scene craft appear in nearly every result, which suggests they are over-retrieving rather than that the dialogue entries are badly written.

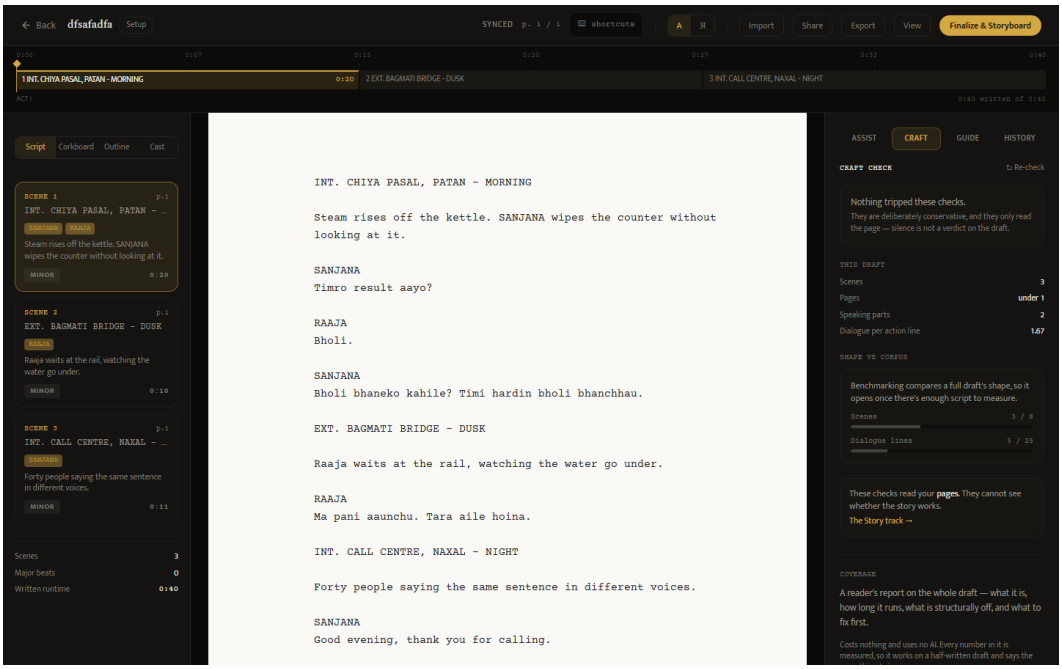

Figure 1. *The panel this measurement supports. Rule-based checks report that nothing was flagged, and state plainly that this is not the same as the draft being finished. The draft's own statistics sit below, and beneath those the corpus comparison, which declines to report until there is enough script to compare against.*

3. Running against live providers

Everything to this point had been verified against a mock AI provider. Running with real credentials found two things, one of which had been costing nothing only because the keys did not exist yet.

The test suite was calling the live API. Its configuration deliberately forces a mock database and pins the payment providers offline, but nobody had done the same for the AI keys. On the day real keys were added, the tests began making billed calls, and a suite that normally finishes in seconds took 26 minutes because the client library retries each failure several times.

The fix is worth recording because the obvious version does not work. Deleting the environment variable has no effect: the configuration loader fills in any variable that is missing, so removing the key invites the real one straight back on the next import. The variable has to be present and invalid.

The cost model was wrong by a factor of twelve. A storyboard image had been estimated at $0.040, assuming the system used DALL-E 3. It does not.

Item	Previously estimated	Measured
One storyboard frame	$0.040	**$0.0033**
24-frame storyboard	$0.96	**$0.08**
One script with a storyboard	$1.32	**~$0.44**

This inverts the pricing lever. Images account for roughly 18% of what a script costs to produce and text generation for 82%, so the control that matters is the length of generated text, not the cap on storyboard frames. It also means a Pro subscriber at Rs 999 covers their own cost at around sixteen scripts a month, which is far above ordinary use.

4. Using the product as a writer would

A complete short film was written inside the system and put through its own tools. This is the cheapest testing available and it found five faults that no fixture had caught, because a fixture only contains what its author already thought of.

1. `FADE IN:` **was counted as a scene.** A 16-scene script reported 17. The extra scene was about a twenty-fifth of a page long, which dragged down the median scene length and meant every corpus percentile shown to a writer had been computed partly against a scene nobody wrote.
2. **The craft panel analysed the wrong text.** Selecting a problem category made the system examine the category's own description rather than the writer's script, then report the result under a heading saying it had been found in their draft, citing a line number that belonged to a sentence they had never typed.
3. **Renaming a scene had no effect on the scene list.** The timeline went on displaying a heading that no longer existed anywhere in the script, and no amount of saving corrected it.
4. **Focus mode did not fill the screen.** The page was drawn 723 pixels tall on a screen 1,274 pixels high.
5. **The editor address was inconsistent.** It was labelled as carrying a project identifier and actually carried a script one.

The second of those is the one worth dwelling on. It had been shipping for some time, it looked entirely plausible, and only a real script revealed it, because with test data the wrong answer and the right answer look equally arbitrary.

5. The assigned deliverables

All four were built during Month 1 and reported then. Each was re-verified this month, and three were extended.

Storyboard generation. One frame per scene, each assigned a shot type from a fixed vocabulary of nine, chosen from the scene's position within its act rather than at random, so a sequence opens wide and tightens toward its turn. Each frame carries a camera note derived from the shot type, cast, time of day and emotional beat, produced without a model call — a field that was previously written as an empty string on every frame the system had ever generated. The writer can override the shot type, edit the note, reorder frames and regenerate any single frame, and a regeneration never overwrites a note the writer has edited. Generation is capped at 24 frames. The cost of that cap was measured this month and is reported in section 3.

Collaboration and version history. Comments anchor to a line and default to the line under the caret, are attributed to their author, and are ordered by position in the script rather than by time, so reading the notes means reading the script in order. Access is granted per project rather than globally, because a person is usually

a writer on their own work and a reader on somebody else's; the three roles are Administrator, Editor and Viewer, and every protected route takes a minimum role that defaults to Editor, so forgetting to mark a route costs a viewer a read and never grants an unintended write. Snapshots are taken on save and coalesced into one per five-minute window. The diff reports ordered hunks with line numbers and two lines of context, replacing a set-based comparison that scored a scene moved between acts as no change at all.

Export system. Four formats: PDF, Word, Final Draft `.fdx`, and a production package combining a title page, the screenplay, a shot list and the storyboard. Devanagari now renders correctly in PDF, using a font bundled under the SIL Open Font License with its provenance recorded; a test fails if the asset is removed. Every export is titled and named after its project, where previously all four downloaded as `script.pdf`.

Import was added alongside it this month. The proposal does not ask for it, but the omission was conspicuous: everything this system is best at is a form of *reading* a screenplay, and all of it was gated behind typing an existing script in again. Final Draft, Fountain, plain text, Word and PDF are accepted. PDF is the only lossy path, and a scanned page is refused with an explanation rather than imported as an empty script.

Subscription tiers. Free, Pro at Rs 999 and Studio at Rs 2,499, with Khalti and eSewa alongside Stripe behind one interface — Stripe alone could not collect from most Nepali cards, which made the billing system untestable against its own market. A payment row is written before the user leaves for the gateway, because a user returns holding only a payment reference, and taking the tier from that returning request would let anyone return claiming Studio. The tiers differ in ways the code enforces rather than only advertises: three active projects on Free, AI generation and several exports gated, and collaborator seats limited to two, five and unlimited, counted against the project owner's plan.

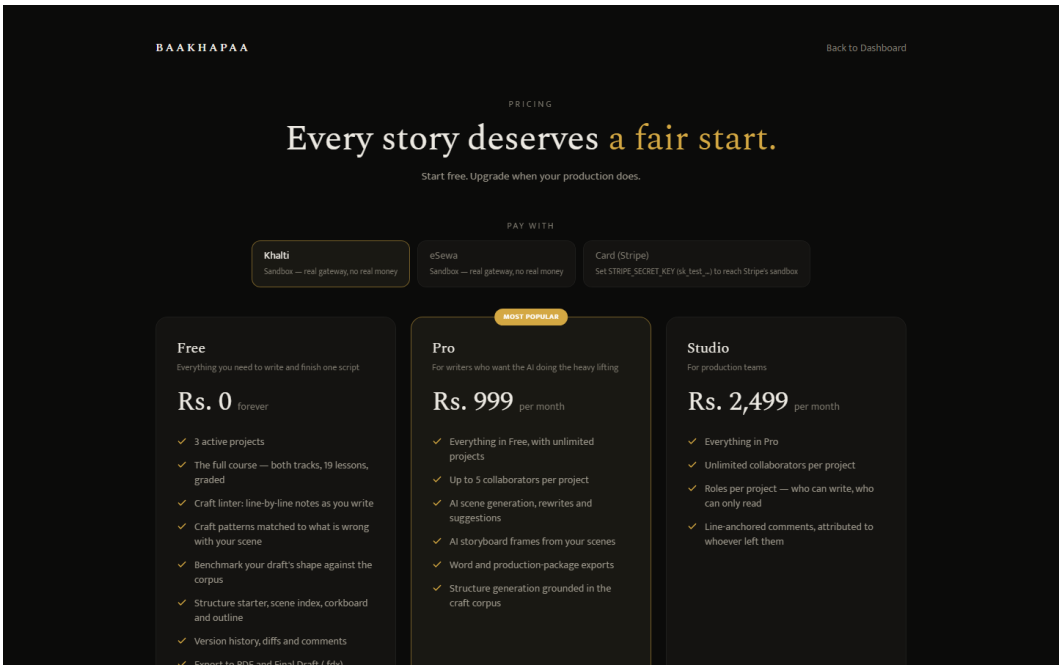

Figure 2. *The three tiers. The pricing page was corrected this month in both directions: Studio had advertised real-time collaboration that was descoped, a seat cap nothing enforced and priority support with no channel, while the Free tier omitted the course, the craft checker, the corpus benchmark, version history and Final Draft export entirely. Tests now pin the specific untrue sentences, so restoring one is a decision made against a failing test.*

6. Testing

Three parts of the backend had no tests at all, and they were close to the inverse of where coverage should have been: the payment webhook, which grants a paid subscription and is the only endpoint that does so without a login; the code deciding which fields a user may change; and the retrieval system, which returns an empty result on failure and therefore reports no error when it breaks.

New test file	What it protects
Payment webhook	That the tier granted comes from the stored payment row and never from the incoming message
Field whitelist	That the two routes taking a raw object cannot be made to write a user identifier or a tier
Version diff	The moved-line case the previous comparison scored as no change
Export fetching	The one place the server fetches a URL a user can influence
Retrieval	Every path, since all of them fail silently by design
Configuration	A guard that every setting the application reads is documented

Two security faults surfaced while writing these. In the version history, the permission check ran after the other error checks rather than before, so the difference between two responses told a signed-in user whether two script versions existed and belonged to the same script, without access to either — every other route in the system returns "not found" precisely to prevent this. Separately, the command palette hid itself from signed-out visitors but still called the server, and the resulting authentication failure signed them out.

7. Work beyond scope

Three additions are recorded for completeness rather than claimed as deliverables.

Streaming. AI requests previously waited for the entire response before showing anything, so a writer asking for a scene watched an indicator for as long as generation took. Scene generation and rewriting now stream as the text is produced, with errors carried inside the stream, since once a response has begun there is no status code left to report a failure with.

Character analysis. A view called Cast gathers each character's dialogue in one place and reports how long their lines run, how much of their wording repeats, and how often they ask a question rather than make a statement. On a test screenplay one character asks in 40% of his lines and another in none — a difference in voice made visible as a number before it is visible on the page. The view also shows the character description the writer entered at project setup, which had been used inside AI prompts and never shown back to them.

Interface. The corkboard and outline moved beside the script rather than replacing it, so scenes can be reordered while the page they belong to is still being read. Page height now matches the pagination rule and standard screenplay margins were applied. Scene headings can be renamed and act durations edited directly on the timeline.

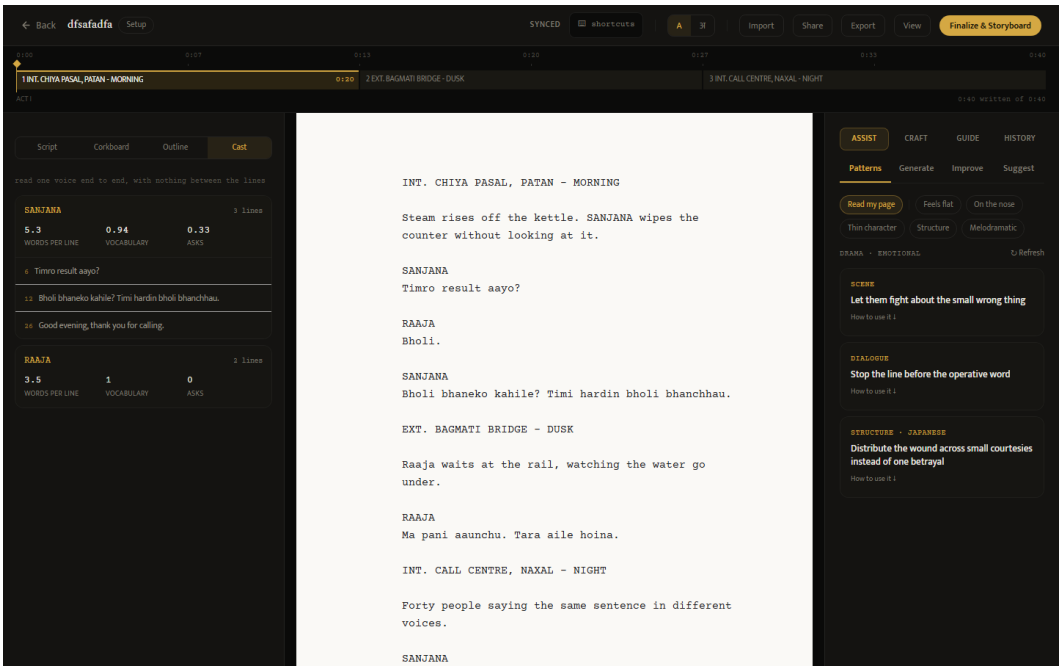

Figure 3. *The Cast view with one character opened. Each line carries its number in the script, and clicking a line moves the cursor to it.*

8. What is not done

The system has never been deployed. It runs on a local database because no cloud database exists, which leaves four schema migrations unapplied. No payment has been processed through any gateway, in any of the three the system now supports. Text generation returns a billing error because the AI account holds no credit, so the paid generation features remain verified against the mock provider only.

Nothing renews automatically. Neither Nepali gateway offers a subscription primitive, so a plan bought through them stops working after thirty days. A reminder script exists but sends nothing until a mail account is configured, and none is.

The terms of use, privacy policy and compliance checklist are unreviewed templates and carry a banner saying so.

9. Month 3

Three items block everything else and are strictly ordered, because each depends on the one before it.

A cloud database is created and the four migrations applied. The email normalisation migration runs first and alone, because it is the only one that can fail on existing data: it adds a uniqueness constraint on lowercased addresses, and two rows differing only in capitalisation will stop it.

The application is then deployed, which exercises the production configuration checks for the first time. Those checks refuse to start on an unset origin list, a demonstration account left enabled, a local database file, or a Devanagari font resolving only to the non-redistributable one. Each was previously a line in a document asking a person to remember something.

Merchant applications to Khalti and eSewa follow deployment, because both require a live website address alongside company registration and a business bank account, and each is reviewed by a person rather than approved automatically.

Two further items follow: a scheduled task for renewal reminders, which needs a mail account; and a pilot with five writers taking real projects from first page to export, which is where the proposal's success measure — one script completed without falling back to manual methods — is demonstrated or disproved.

10. Decisions that cannot be settled by building

Whether Rs 999 holds. The measured cost of a script makes the price safe on margin and questionable on market: it was set against international screenwriting tools, while the subscription price Nepali users are anchored to sits nearer Rs 499. An annual option would also convert twelve monthly opportunities to lapse into one, which matters because neither Nepali gateway renews.

Invite-only or open launch. One developer cannot absorb open registration and a defect queue simultaneously. An invitation period would also make the pilot and the launch the same activity rather than two.

Encryption of script text. Screenplays are stored without application-level encryption. This is now stated truthfully in the privacy documentation rather than implied otherwise, but stating it is not resolving it. Adding it is a design decision rather than a feature, because it changes what diffing, search and export can do, and retrofitting it after a pilot is considerably harder than deciding it before one.

Who reviews the legal documents. A Nepal-qualified lawyer is required. This is the one item on the project that cannot be compressed by working harder, because it depends on somebody else's calendar.

Figures were captured from the running system using `baakhapaa-frontend/scripts/capture-screenshots.mjs`.